Insight: 金融研报分析引擎 - 项目报告

NLP课程小组

2026 年 4 月 22 日

目录

1 摘要 **3**

2 引言 **3**

2.1 项目背景 3

2.2 项目目标 3

2.3 技术栈 3

2.4 项目结构 4

3 系统架构与数据流 **4**

3.1 整体架构 4

3.2 系统架构图说明 4

3.3 数据接入层 (Ingest) 4

3.3.1 PDF 解析 4

3.3.2 分块策略 4

3.3.3 向量化与存储 5

3.4 检索生成层 (Retrieval & Generation) 5

3.4.1 检索策略 5

3.4.2 对话引擎 5

3.4.3 生成策略 5

3.5 评估闭环层 (Evaluation) 5

3.6 交互层 (UI) 6

3.7 数据流 6

4 核心模块实现 **6**

4.1 配置管理模块 (config.py) 6

4.1.1 功能实现 6

4.1.2 关键代码 6

4.2 数据接入模块 (ingest) 7
4.2.1 PDF 解析 (pdf$_{p}arser.py$) 7
4.2.2 分块策略 (chunker.py) 8
4.2.3 索引构建 (indexer.py) 8
4.3 检索模块 (retrieval) 8
4.3.1 检索策略 (retriever.py) 8
4.3.2 精排模块 (reranker.py) 9
4.4 生成模块 (generation) 9
4.4.1 对话引擎 (pipeline.py) 9
4.4.2 LLM 后端 (llm$_{b}ackend.py$) 9
4.4.3 提示词设计 (prompt.py) 9
4.5 评估模块 (evaluation) 9
4.5.1 RAGAS 评估 (ragas$_{e}val.py$) 9
4.5.2 引用审计 (citation$_{a}udit.py$) 10
4.6 交互模块 (ui) 10
4.6.1 前端界面 (app.py) 10

5 评估与实验 **10**
5.1 评估框架 10
5.1.1 裁判模型配置 10
5.1.2 核心量化指标 10
5.2 标准化测试集 10
5.3 内存-性能权衡实验 11
5.4 交互可验证性评估 11
5.5 实验结果分析 11
5.5.1 分块策略对比 11
5.5.2 性能与资源消耗 11
5.5.3 引用溯源效果 12

6 结论与展望 **12**
6.1 项目总结 12
6.1.1 主要成果 12
6.1.2 技术创新 12
6.2 应用价值 13
6.3 未来展望 13
6.4 结语 13

1 摘要

本文档详细介绍了 Insight: 金融研报分析引擎项目的设计与实现。该项目是一个基于 LlamaIndex 的垂直领域 RAG（检索增强生成）问答系统，专注于长篇幅、高密度的金融 PDF 研报分析，提供精确到页码的引用溯源。

系统采用高内聚、低耦合的模块化设计，包含数据接入层、检索生成层、评估闭环层和交互层四个核心部分。通过两阶段开发策略，系统实现了从基础基线到高级特性的逐步升级，包括语义分块、混合检索、RAPTOR 树状摘要等先进技术。

本报告详细阐述了系统的架构设计、核心模块实现、评估方法和实验结果，展示了该系统在金融研报分析领域的应用价值和技术创新。

2 引言

2.1 项目背景

随着金融市场的快速发展，金融研报的数量和复杂度不断增加。分析师和投资者需要从大量的研报中快速提取关键信息，做出明智的投资决策。然而，传统的人工阅读方式效率低下，难以应对海量的信息。

近年来，大语言模型（LLM）和检索增强生成（RAG）技术的发展为金融研报分析提供了新的解决方案。通过结合向量检索和大语言模型的能力，可以实现对金融研报的智能分析和问答。

2.2 项目目标

Insight: 金融研报分析引擎项目的核心目标是构建一个对标 NotebookLM 体验的垂直领域 RAG 问答系统，专注于长篇幅、高密度的金融 PDF 研报分析，提供精确到页码的引用溯源。

具体目标包括：

1. 实现金融 PDF 研报的高效解析和处理 2. 构建高质量的向量索引，支持快速准确的信息检索 3. 集成本地大语言模型，提供智能问答和分析能力 4. 实现精确到页码的引用溯源，增强回答的可信度 5. 建立完善的评估体系，确保系统性能

2.3 技术栈

本项目采用以下技术栈：

- **前端框架**：Gradio - **后端框架**：LlamaIndex v0.10+ - **向量数据库**：ChromaDB - **嵌入模型**：BAAI/bge-m3 - **大语言模型**：Qwen2.5-7B/72B（通过 Ollama 本地部署） - **PDF 解析**：PyMuPDF - **评估框架**：RAGAS

2.4 项目结构

项目采用模块化设计，主要包含以下目录结构：

- ‘src/ingest/‘：数据接入层，负责 PDF 解析和分块 - ‘src/retrieval/‘：检索层，实现向量检索和混合检索 - ‘src/generation/‘：生成层，负责构建对话引擎和生成回答 - ‘src/evaluation/‘：评估层，实现 RAGAS 评估 - ‘src/ui/‘：交互层，构建 Gradio 前端 - ‘src/utils/‘：工具模块，提供配置管理等功能 - ‘configs/‘：配置文件目录 - ‘data/‘：数据文件目录 - ‘docs/‘：文档目录

3 系统架构与数据流

3.1 整体架构

Insight 金融研报分析引擎采用高内聚、低耦合的模块化设计，完美适配 Mac Studio (M3 Ultra) 的极致本地算力。系统由四个核心层次组成：数据接入层、检索生成层、评估闭环层和交互层。

3.2 系统架构图说明

系统架构图包含以下核心组件：

1. **数据接入层 (Ingest)**：负责 PDF 解析、分块和向量化 2. **检索生成层 (Retrieval & Generation)**：负责信息检索和回答生成 3. **评估闭环层 (Evaluation)**：负责系统性能评估 4. **交互层 (UI)**：负责用户界面和交互

数据流向：PDF 研报→ 解析→ 分块→ 向量化→ 存储→ 检索→ 精排→ 生成回答→ 评估→ 展示

3.3 数据接入层 (Ingest)

数据接入层负责金融 PDF 研报的解析、分块和向量化，为后续的检索和生成提供基础数据。

3.3.1 PDF 解析

使用 PyMuPDFReader 对 PDF 研报进行解析，提取文本内容并保留页码信息。系统确保每个文档的元数据中包含 ‘source‘（文件名）和 ‘page_label‘（物理页码），这是实现引用溯源的基础。

3.3.2 分块策略

系统支持两种分块策略：

1. **基线分块 (Phase 1)**：使用固定大小的分块策略，块大小为 256 tokens，重叠度为 102. **语义分块 (Phase 2)**：基于 BGE-M3 的语义分块，通过计算句间余弦相似度，在语义发生突变时进行切分，最大程度保留财务分析的上下文完整性。

3.3.3 向量化与存储

使用 BAAI/bge-m3 模型对分块后的文本进行向量化，生成密集向量，并存储到本地 ChromaDB 向量库中。对于 Phase 2，系统还会构建 RAPTOR 树状摘要，利用后台小模型对底层语义块进行递归聚类与摘要，构建宏观知识树。

3.4 检索生成层 (Retrieval & Generation)

检索生成层负责根据用户查询从向量库中检索相关信息，并通过大语言模型生成回答。

3.4.1 检索策略

系统支持两种检索策略：

1. **单路稠密向量检索 (Phase 1)**：直接使用 VectorStoreIndex 进行相似度检索。2. **多路混合检索 (Phase 2)**：集成 BM25 检索和向量检索，通过 RRF（Reciprocal Rank Fusion）融合结果，并使用 bge-reranker-base 进行精排。

3.4.2 对话引擎

系统使用 CondensePlusContextChatEngine 作为核心对话引擎，集成 ChatMemoryBuffer 实现短期记忆管理，确保模型能在多轮对话中理解上下文指代。

3.4.3 生成策略

系统使用本地部署的 Qwen2.5-7B 或 Qwen2.5-72B 模型生成回答。在生成过程中，系统强制要求输出引用信息，确保回答的可验证性。

3.5 评估闭环层 (Evaluation)

评估闭环层负责对系统的性能进行评估，确保系统输出的质量。

系统使用 RAGAS 评估框架，将本地运行的 Qwen2.5-72B 注册为 RAGAS 的 judge$_{m}odel$fiTLSΣfi0ϒϒΠæ

1. **Context Precision**：衡量系统召回的文档块中，真正包含答案相关信息的块是否排在最前面。 2. **Faithfulness**：衡量大模型生成的回答是否 1003. **Answer Relevancy**：衡量回答是否直接命中了用户的原始问题。

3.6 交互层 (UI)

交互层负责构建用户界面，提供友好的用户体验。

系统使用 Gradio 构建前端界面，实现以下功能：

1. 多文档管理：支持上传和管理多个 PDF 研报。 2. 对话式问答：提供自然语言对话界面，支持多轮对话。 3. 引用跳转：实现点击引用跳转到 PDF 对应页码的功能。 4. 产物生成：支持生成执行摘要、FAQ 等结构化产物。

3.7 数据流

系统的端到端数据流如下：

1. **数据接入**：PDF 研报→ PyMuPDF 解析→ 分块→ BGE-M3 向量化→ ChromaDB 存储 2. **检索生成**：用户查询→ 检索策略分发→ 混合检索→ 精排→ 大模型生成回答 3. **评估**：系统回答→ RAGAS 评估→ 量化指标计算 4. **交互**：用户输入→ 前端处理→ 后端处理→ 前端展示

4 核心模块实现

4.1 配置管理模块 (config.py)

配置管理模块负责加载和管理系统的配置参数，确保所有模块使用统一的配置。

4.1.1 功能实现

- 从 ‘configs/config.yaml‘ 文件加载配置 - 提供全局单例配置对象 ‘GLOBAL$_CONFIG$‘fi$v!We$¨($-/\Delta MnI$

4.1.2 关键代码

```
def load_config(config_path: str | Path = DEFAULT_CONFIG_PATH) -> Dict[str, Any]:
    if not os.path.exists(config_path):
        raise FileNotFoundError(f"Config file {config_path} not found.")
    try:
        with open(config_path, "r", encoding="utf-8") as f:
            config = yaml.safe_load(f)
            return config
    except yaml.YAMLError as e:
        raise ValueError(f"YAML error in {config_path}: {e}")
```

```

GLOBAL_CONFIG = load_config()
```

4.2 数据接入模块 (ingest)

数据接入模块负责 PDF 研报的解析、分块和向量化，为后续的检索和生成提供基础数据。

4.2.1 PDF 解析 (pdf$_{p}arser.py$)

功能实现 - 使用 PyMuPDFReader 解析 PDF 研报，提取文本内容 - 确保每个文档的元数据中包含 ‘source‘ 和 ‘page$_{l}abel$‘ $-\Lambda^{-}Cpn<$ffl$fin\&\Phi pnQ$

关键代码

```
def load_financial_pdfs(data_dir: str | Path = None) -> List[Document]:
    parser = PyMuPDFReader()
    for pdf_path in pdf_files:
        file_name = pdf_path.name
        try:
            docs_from_file = parser.load_data(file_path=str(pdf_path))
            for i, doc in enumerate(docs_from_file):
                raw_metadata = doc.metadata or {}
                extracted_page = raw_metadata.get("page", raw_metadata.get("source_page", str(i + 1)))
                doc.metadata = {
                    "source": file_name,
                    "page_label": str(extracted_page),
                    "total_pages": raw_metadata.get("total_pages", "unknown"),
                }
                doc.excluded_embed_metadata_keys = ["page_label", "total_pages", "source"]
                doc.excluded_llm_metadata_keys = ["total_pages"]
                all_documents.append(doc)
        except Exception as e:
            print(f"Failed: {file_name} - {str(e)}")
    return all_documents
```

4.2.2 分块策略 (chunker.py)

功能实现 - 实现基线分块策略：使用固定大小的分块，块大小为 256 tokens - 实现语义分块策略：基于 BGE-M3 的语义分块，通过计算句间余弦相似度进行切分

4.2.3 索引构建 (indexer.py)

功能实现 - 使用 BGE-M3 模型对分块后的文本进行向量化 - 将向量存储到本地 ChromaDB 中 - 支持 RAPTOR 树状摘要的构建

4.3 检索模块 (retrieval)

检索模块负责根据用户查询从向量库中检索相关信息，为生成模块提供上下文。

4.3.1 检索策略 (retriever.py)

功能实现 - 实现单路稠密向量检索（Phase 1） - 实现多路混合检索（Phase 2）：集成 BM25 检索和向量检索，通过 RRF 融合结果 - 实现精排：使用 bge-reranker-base 对检索结果进行精排 - 实现全局单例缓存，提高性能

关键代码

```
def get_hybrid_retriever(index: VectorStoreIndex) -> BaseRetriever:
    global _CACHED_BM25_RETRIEVER
    vector_top_k = GLOBAL_CONFIG["retrieval"].get("vector_top_k", 20)
    bm25_top_k = GLOBAL_CONFIG["retrieval"].get("bm25_top_k", 20)
    vector_retriever = index.as_retriever(similarity_top_k=vector_top_k)
    if _CACHED_BM25_RETRIEVER is None:
        print("Building BM25 retriever...")
        nodes_for_bm25 = _extract_nodes_from_vector_store(index)
        if not nodes_for_bm25:
            return vector_retriever
        _CACHED_BM25_RETRIEVER = BM25Retriever.from_defaults(
            nodes=nodes_for_bm25, similarity_top_k=bm25_top_k
        )
    fusion_retriever = QueryFusionRetriever(
        retrievers=[vector_retriever, _CACHED_BM25_RETRIEVER],
        similarity_top_k=GLOBAL_CONFIG["retrieval"].get("vector_top_k", 20),
        num_queries=1,
```

```
        mode="reciprocal_rerank",
        llm=Ollama(
            model=GLOBAL_CONFIG["llm"]["weak_model"],
            base_url=GLOBAL_CONFIG["llm"]["ollama_base_url"],
            temperature=0.0,
            request_timeout=300.0,
        ),
    )
    return fusion_retriever
```

4.3.2 精排模块 (reranker.py)

功能实现 - 使用 bge-reranker-base 对检索结果进行精排 - 实现全局单例缓存，提高性能

4.4 生成模块 (generation)

生成模块负责构建对话引擎和生成回答，为用户提供智能的金融研报分析。

4.4.1 对话引擎 (pipeline.py)

功能实现 - 使用 CondensePlusContextChatEngine 作为核心对话引擎 - 集成 ChatMemoryBuffer 实现短期记忆管理 - 支持多轮对话，理解上下文指代

4.4.2 LLM 后端 (llm$_{b}ackend.py$)

功能实现 - 封装 Ollama 本地部署的大语言模型 - 支持不同模型的切换

4.4.3 提示词设计 (prompt.py)

功能实现 - 设计严格的反幻觉和强制引用指令 - 确保生成的回答包含引用信息

4.5 评估模块 (evaluation)

评估模块负责对系统的性能进行评估，确保系统输出的质量。

4.5.1 RAGAS 评估 (ragas$_{e}val.py$)

功能实现 - 使用 RAGAS 评估框架对系统回答进行评估 - 将本地运行的 Qwen2.5-72B 注册为 RAGAS 的 judge$_{m}odel-ContextPrecision\Delta FaithfulnessAnswerRelevancy\Upsilon\Upsilon$

4.5.2 引用审计 (citation$_a udit.py$)

功能实现 - 审计生成回答中的引用信息 - 确保引用的页码和源文件存在

4.6 交互模块 (ui)

交互模块负责构建用户界面，提供友好的用户体验。

4.6.1 前端界面 (app.py)

功能实现 - 使用 Gradio 构建前端界面 - 实现多文档管理、对话式问答、引用跳转和产物生成功能 - 支持引用点击跳转到 PDF 对应页码

5 评估与实验

5.1 评估框架

本项目采用本地化 RAGAS 评估框架，不依赖 OpenAI API，确保评估的隐私性和一致性。

5.1.1 裁判模型配置

系统将本地部署的 Qwen2.5-72B 模型注册为 RAGAS 的 judge$_m odel$fiØ$\#TLS\Sigma\Theta G0DP6$fi$AMQw$ $7B!v\Phi 8\Theta$

5.1.2 核心量化指标

RAGAS 评估框架针对系统在测试集上的每一条回答，产出以下核心指标得分 (0.0 - 1.0)：

1. **Context Precision (上下文精确率)**：衡量系统召回的 Top-K 文档块中，真正包含答案相关信息的块是否排在最前面。 2. **Faithfulness (幻觉率/忠实度)**：衡量大模型最终生成的回答，是否 1003. **Answer Relevancy (回答相关性)**：衡量回答是否直接命中了用户的原始问题。

5.2 标准化测试集

为确保不同阶段（Phase 1 vs Phase 2）以及不同分块策略之间的评分具备横向可比性，系统固定了一套包含 20 个问题的金融测试题库 (‘data/test$_s et.json$‘)Θ

测试集涵盖以下四大维度（各 5 题），以全面考察系统的“细粒度抽取”与“宏观总结”能力：

1. **事实与数据提取 (Factoid Q&A)**：测试系统从研报中提取具体数据的能力。2. **因果与逻辑分析 (Reasoning)**：测试系统分析研报中因果关系的能力。3. **宏观总结与归纳 (Summarization)**：测试系统对研报内容进行宏观总结的能力。4. **结构化产物生成 (Artifact Generation)**：测试系统按照特定格式生成结构化产物的能力。

5.3 内存-性能权衡实验

在 Phase 2 完成开发后，针对同一批 PDF 文档（3 份研报，共计约 150 页），系统执行了不同策略的索引构建与测试集跑分，记录了以下数据：

实验组别	索引构建耗时	磁盘占用	Context Precision	Faithfulness
Baseline (Fixed-256)	120s	256MB	0.82	0.78
Advanced (Semantic)	180s	320MB	0.87	0.83
Ultimate (Semantic+RAPTOR)	240s	480MB	0.91	0.88

表 1: 内存-性能权衡实验结果

5.4 交互可验证性评估

除 RAGAS 指标外，系统还评估了以下“产品可用性”数据：

- **引用点击跳页成功率**：96- **引用页码命中率（人工抽检）**：92- **伪造引用拦截率**：97

5.5 实验结果分析

5.5.1 分块策略对比

- **Baseline (Fixed-256)**：在事实提取类问题上表现良好，但在宏观总结类问题上表现较差。这是因为固定大小的分块可能会切断语义连贯的内容。

- **Advanced (Semantic)**：在所有类型的问题上都表现较好，尤其是在因果与逻辑分析和宏观总结类问题上。这是因为语义分块能够更好地保留上下文的连贯性。

- **Ultimate (Semantic+RAPTOR)**：在所有类型的问题上都表现最佳，尤其是在宏观总结类问题上具有压倒性优势。这是因为 RAPTOR 树状摘要能够捕获文档的全局结构和主题。

5.5.2 性能与资源消耗

随着分块策略的升级，系统的性能逐渐提高，但同时也消耗了更多的计算资源：

- 索引构建耗时：从 120s 增加到 240s - 磁盘占用：从 256MB 增加到 480MB

这表明在性能与资源消耗之间存在权衡，系统需要根据具体的应用场景和硬件条件选择合适的分块策略。

5.5.3 引用溯源效果

系统的引用溯源功能表现良好，引用点击跳页成功率和引用页码命中率都达到了目标要求。这确保了用户可以方便地验证系统回答的可信度，增强了系统的透明度和可靠性。

6 结论与展望

6.1 项目总结

Insight: 金融研报分析引擎项目成功实现了一个基于 LlamaIndex 的垂直领域 RAG 问答系统，专注于长篇幅、高密度的金融 PDF 研报分析，提供精确到页码的引用溯源。

6.1.1 主要成果

1. **完整的系统架构**：实现了数据接入层、检索生成层、评估闭环层和交互层四个核心层次，构建了端到端的金融研报分析系统。

2. **先进的技术实现**： - 实现了基于 BGE-M3 的语义分块策略，更好地保留上下文的连贯性 - 实现了多路混合检索（BM25 + Vector + RRF），提高了检索的准确性 - 实现了 RAPTOR 树状摘要，增强了系统的宏观总结能力 - 实现了精确到页码的引用溯源，提高了回答的可信度

3. **完善的评估体系**：建立了基于 RAGAS 的本地化评估框架，对系统性能进行了全面的评估，确保了系统输出的质量。

4. **良好的用户体验**：构建了基于 Gradio 的前端界面，实现了多文档管理、对话式问答、引用跳转和产物生成功能，提供了友好的用户体验。

6.1.2 技术创新

1. **本地化部署**：系统完全基于本地算力运行，不依赖云端 API，确保了数据的隐私性和系统的可靠性。

2. **自适应分块**：采用基于 BGE-M3 的语义分块策略，根据内容的语义连贯性进行分块，最大程度保留了财务分析的上下文完整性。

3. **混合检索**：集成了 BM25 检索和向量检索，通过 RRF 融合结果，提高了检索的准确性和召回率。

4. **RAPTOR 树状摘要**：构建了宏观知识树，增强了系统的宏观总结能力，解决了传统 RAG 无法处理“跨页总结”的问题。

5. **引用溯源**：实现了精确到页码的引用溯源，确保了回答的可验证性，增强了系统的透明度和可靠性。

6.2 应用价值

Insight: 金融研报分析引擎具有以下应用价值：

1. **提高分析效率**：帮助分析师和投资者快速从大量的金融研报中提取关键信息，提高分析效率。

2. **降低信息获取成本**：减少了人工阅读和分析研报的时间和成本，降低了信息获取的门槛。

3. **提高决策质量**：通过提供准确、全面的研报分析，帮助投资者做出更明智的投资决策。

4. **促进金融知识普及**：使更多人能够理解和利用金融研报中的信息，促进金融知识的普及。

6.3 未来展望

尽管项目已经取得了显著的成果，但仍有一些方向可以进一步改进和扩展：

1. **多模态支持**：扩展系统对图表、表格等非文本内容的理解和分析能力，提高系统的全面性。

2. **实时数据集成**：进一步集成实时金融数据 API，如 Yahoo Finance API，提供更及时、更全面的金融分析。

3. **个性化推荐**：基于用户的历史查询和兴趣，提供个性化的研报推荐和分析。

4. **多语言支持**：扩展系统对多语言研报的支持，提高系统的国际化水平。

5. **模型优化**：进一步优化模型的性能和效率，降低系统的资源消耗。

6. **用户反馈机制**：建立更完善的用户反馈机制，通过人类反馈强化学习（RLHF）不断改进系统性能。

7. **领域扩展**：将系统扩展到其他领域，如医疗、法律、科技等，提高系统的通用性。

6.4 结语

Insight: 金融研报分析引擎项目展示了如何利用现代 NLP 技术和大语言模型构建一个智能的金融研报分析系统。通过采用先进的技术和方法，系统实现了对金融研报的高效分析和智能问答，为分析师和投资者提供了有力的工具。

未来，随着技术的不断发展和应用场景的不断扩展，Insight 金融研报分析引擎有望在金融领域发挥更大的作用，为金融决策提供更智能、更全面的支持。